

④ Semantic error: These are the errors that occur when statements are not understandable by the compiler.
eg: $a + b = c$ instead of $c = a + b$

⑤ Linker error: When an executable file is not generated, it is called a linker error.
This happens when the linker is not able to link multiple files.
eg: Function has been called in main but is not defined

Removing errors in a program is called de-bugging.

Types of error:

- ① Syntax error: These error occur when the programmer doesn't follow the rule of the language or typo error.

Most common syntax errors are:-

- ① Missing semicolon ';'
- ② Missing bracket.
- ③ Missing variable declaration etc.

- ⑤ logical error: This error occurs after successful compilation of the program.

The occurrence of this error mainly depended on logical thinking of developer.

eg: $x = 5$ and $x = 5$
① $(a+b)/2$, $a + b/2$

- ③ Runtime Error: This error occurs during execution time even after successful compilation, when the CPU not able to perform the operation. It is called runtime error.

eg: Divide by '0' error

② It should be a logical name.

③ Operator- Operator are symbol that perform operation on operands.
eg: $a+b$, $a-c$, $a*b$, $+$, $-$ are operator

There 7 types of operators:
eg: arithmetic, logical, bitwise etc.

④ Special Symbol:- Some symbol are use for some special & function, work
eg: $\#$, $;$, $:$, $\{ \}$, $< >$,

⑤ Strings - Collection of character is called a string.

It is enclosed in "double quote" "

Operands:- Operands are identifier on which operation is been perform by the operator.

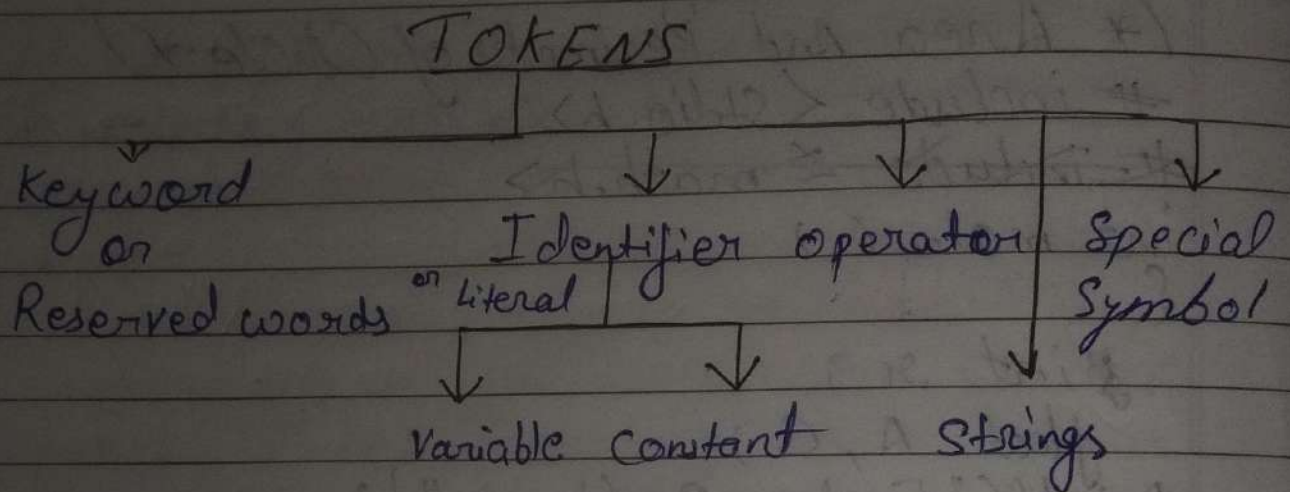
$$a = \underbrace{b + c}_{\text{operator}}$$

↑
operands

~~FE~~ Error:-

Error are fault, mistake or bugs that occur in a program which make a behavior of program abnormal.

Tokens



Tokens are smallest component in C language that has meaning.

C programming is made up of has a following tokens

- ① Keywords - C language has 32 keyword.
 - keywords has fix pre-define meaning, there name & function cannot be changed
 - eg: printf, if, switch, case, while, for etc
- ② Identifier - Identifier are user define names given to variable, constant etc.
 - * Rules to be follow:
 - ① It cannot be a keyword
 - ② No blank space is allow
 - ③ It cannot start with a number.
 - ④ It is combination of alphabet, digits and underscore. Only special special symbol allow is -
 - ⑤ no other symbol allowed.
 - ⑥ The maximum length of an identifier is 32.
 - ⑦ It is case sensitive.

```
/* My first C Prog */
```

```
#include <Stdio.h>
```

```
int main ( )
```

```
{
    Printf("Hello World");
    return 0;
}
```

Q Write a C program to add two numbers.

Ans: /* Sum of two number */

```
#include <Stdio.h>
```

```
int main ( )
```

```
{
    int n1, n2, Sum;
    Printf("Enter 2 no");
    Scanf("%d%d", &n1, &n2);
    Sum = n1 + n2;
    Printf("The answer is %d", Sum);
}
```

variable
location

format specifier

new
line

n1	15
n2	4
Sum	19

Enter 2 no. - 15

4

The answer is 19

Structure of C program :-

Comment Line (Optional)

File Inclusion (Mandatory)

Global Declaration (Optional)
(Preprocessor Directive)

Main function (Mandatory)
Local declaration & Body

User Defined function (Optional)

Chapter 3 - Conditional Instructions

Sometimes we want to watch comedy videos on YouTube if the day is Sunday.

Sometimes we order junk food if it is our friend's birthday in the hostel.

You might want to buy an Umbrella if it's raining and you have the money.

You order the meal if dal or your favorite bhindi is listed on the menu.

All these are decisions which depends on a condition being met.

In C language too, we must be able to execute instructions on a condition(s) being met.

Decision Making Instructions in C

→ If - else Statement

→ Switch Statement

If - else Statement

The syntax of an If - else Statement in C looks like :

```
if (condition to be checked) {  
    Statements - if - condition - true;  
}  
else {  
    Statements - if - condition - false;  
}
```


Code example:

```
int a = 23;
```

```
if (a > 18) {  
    printf("You can drive\n");  
}
```

Note that else block is not necessary but optional.

Relational Operators in C

Relational operators are used to evaluate conditions (true or false) inside the if statements. Some examples of relational operators are :-

$=$	$>=$	$>$	$<$	$<=$	$!=$
↓	↓				↓
equals	greater than or equal to				not equal to

Important note :- '=' is used for assignment whereas '==' is used for equality check.

The condition can be any valid expression. In C a non-zero value is considered to be true.

Logical Operators

&&, || and ! are three logical operators in C. These are read as "AND", "OR" and "NOT". They are used to provide logic to our C programs.

Usage of Logical Operators:

(i) $\& \& \rightarrow$ AND $\rightarrow$ is true when both the conditions are true

"1 and 0" is evaluated as false.

"0 and 0" is evaluated as false.

"1 and 1" is evaluated as true.

(ii) $\parallel \rightarrow$ OR $\rightarrow$ is true when at least one of the conditions is true. $(1 \text{ or } 0 \rightarrow 1)(1 \text{ or } 1 \rightarrow 1)$

(iii) $! \rightarrow$ returns true if given false and false if given true

$!(3 == 3) \rightarrow$ evaluates to false

$!(3 > 30) \rightarrow$ evaluates to true.

As the number of conditions increases, the level of indentation increases. This reduces readability. Logical operators come to rescue in such cases.

else if clause

Instead of using multiple if statements, we can also use else if along with if thus forming an if-else-if-else ladder.

```
if {  
    // statements;  
}
```

```
else if {  
    ...  
}
```

```
else {  
    ...  
}
```

Using if - else if - else reduces indents

The last "else" is optional

Also there can be any number of 'else if'

Last else is executed only if all conditions fail.

Operator precedence

Priority	Operator
1 st	!
2 nd	*, /
3 rd	+, -
4 th	<, >, <
5 th	=, =, !
6 th	&&
7 th	
8 th	=

Conditional Operators

A short hand "if - else" can be written using the Conditional or ternary operators

Condition ? expression-if-true ; expression-if-false

Ternary operators

Switch Case Control Instruction

Switch-Case is used when we have to make a choice between number of alternatives for a given variable.

```
Switch (integer-expression)
{
```

```
    Case C1:
```

```
        Code;
```

```
    Case C2:
```

```
        Code;
```

```
    Case C3:
```

```
        Code;
```

```
    default:
```

```
        Code;
```

```
}
```

$C_1, C_2 \text{ \& } C_3 \rightarrow \text{Constants}$

$\text{Code} \rightarrow \text{Any valid C Code.}$

The value of integer-expression is matched against $C_1, C_2, C_3, \dots$. If it matches any of these cases, that case along with all subsequent "case" and "default" statements are executed.

* Quick Quiz: Write a program to find grade of a student given his marks based on below:

$\rightarrow 90 - 100 \rightarrow A$ $\rightarrow < 70 \rightarrow F$.

$\rightarrow 80 - 90 \rightarrow B$

$\rightarrow 70 - 80 \rightarrow C$

$\rightarrow 60 - 70 \rightarrow D$

Important Notes

- 1> We can use switch-case statements even by writing cases in any order of our choice (not necessarily ascending)
- 2> char values are allowed as they can be easily evaluated to an integer
- 3> A switch can occur within another but in practice this is rarely done.

1. Square Pattern.

```
#include <stdio.h>
int main()
{
    int n;
    printf("Enter a number: ");
    scanf("%d", &n);

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            printf("* ");
        }
        printf("\n");
    }
}
```

2. Hollow Square.

```
#include <stdio.h>
int main() {
    int n;
    printf("Enter the size of the square");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (i == 0 || i == n - 1 || j == 0 || j == n - 1) {
                printf("* ");
            } else {
                printf(" ");
            }
        }
        printf("\n");
    }
}
```

③ Triangle.

```
int n;  
printf("Enter the no. of rows: ");  
scanf("%d", &n);  
  
for (int i=1; i<=n; i++) {  
    for (int j=1; j<=i; j++) {  
        printf("* ");  
    }  
    printf("\n");  
}
```

④ Inverted Triangle.

```
int n;  
printf("Enter no. of rows: ");  
scanf("%d", &n);  
  
for (int i=n; i>=1; i--) {  
    for (int j=1; j<=i; j++) {  
        printf("* ");  
    }  
    printf("\n");  
}
```


5. Pyramid.

```

int n;
printf("Enter the number of rows: ");
scanf("%d", &n);

for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n - i; j++)
        printf(" ");
    for (int k = 1; k <= 2 * i - 1; k++)
        printf("*");
    printf("\n");
}

```

6. Inverted Pyramid.

```

int n;
printf("Enter the no. of rows: ");
scanf("%d", &n);

for (int i = n; i >= 1; i--) {
    for (int k = 1; k <= n - i; k++)
        printf(" ");
    for (int j = 1; j <= 2 * i - 1; j++)
        printf("*");
    printf("\n");
}

```

Half Diamond.

```
int n;
```

```
printf("Enter the no. of rows ");
```

```
scanf("%d", &n);
```

```
for (int i = 1; i <= n; i++) {
```

```
    for (int j = 1; j <= i; j++) {
```

```
        printf(" * ");
```

```
    }
```

```
    printf("\n");
```

```
}
```

```
for (int i = n - 1; i >= 1; i--) {
```

```
    for (int j = n; j >= n - i + 1; j--) {
```

```
        printf(" * ");
```

```
    }
```

```
    printf("\n");
```

```
}
```

```
}
```


Diamond

```
#include <stdio.h>
```

```
int main () {
```

```
    int n;
```

```
    printf ("Enter the no. of rows: ");
```

```
    scanf ("%d", &n);
```

```
    for (int i = 1; i <= n; i++) {
```

```
        for (int j = 1; j <= n - i; j++)
```

```
            printf (" ");
```

```
        for (int k = 1; k <= 2 * i - 1; k++)
```

```
            printf ("*");
```

```
        printf ("\n");
```

```
    }
```

```
    for (int i = n - 1; i >= 1; i--) {
```

```
        for (int j = 1; j <= n - i; j++)
```

```
            printf (" ");
```

```
        for (int k = 1; k <= 2 * i - 1; k++)
```

```
            printf ("*");
```

```
        printf ("\n");
```

```
    }
```

```
}
```

← → C

Update

zsh

C square.c U x

CHAT TERMINAL

1 #include<stdio.h>

2 int main() {

3 int n;

4 printf("Enter a number: ");

5 scanf("%d", &n);

6

7 for(int i=1; i<=n; i++) {

8 for(int j=1; j<=n; j++) {

9 printf("* ");

10 }

11 printf("\n");

12

13 }

14 }

• abhinavsharma@Abhinavs-MacBook-Air C % cd "/Users/abhinavsharma/Comp. Language/C/" && gcc square.c -o square && "/Users/abhinavsharma/Comp. Language/C/"square

Enter a number: 6

* * * * *

* * * * *

* * * * *

* * * * *

* * * * *

* * * * *

• abhinavsharma@Abhinavs-MacBook-Air C %

main*

Ln 8, Col 22

Spaces: 4

UTF-8

LF

{ } C

Go Live

macos-clang-arm64

Prettier

← → C

Update

zsh

C square.c U

C hollowSquare.c U

hollowSquare.c

main()

```
1 #include <stdio.h>
2 int main() {
3     int n;
4     printf("Enter the size of the square: ");
5     scanf("%d", &n);
6
7     for (int i = 0; i < n; i++) {
8         for (int j = 0; j < n; j++) {
9             if (i == 0 || i == n - 1 || j == 0 || j == n - 1) {
10                 printf("* ");
11             } else {
12                 printf(" ");
13             }
14         }
15         printf("\n");
16     }
17
18     return 0;
19 }
```

CHAT

TERMINAL

abhinavsharma@Abhinavs-MacBook-Air C % cd "/Users/abhinavsharma/Comp. Language/C/" && gcc hollowSquare.c -o hollowSquare && "/Users/abhinavsharma/Comp. Language/C/"hollowSquare

Enter the size of the square: 5

* * * * *

* *

* *

* *

* * * * *

abhinavsharma@Abhinavs-MacBook-Air C %

OUTPUT

Filter (e.g. text, textcl...)

Code

[Running] cd "/Users/abhinavsharma/Comp. Language/C/" && gcc square.c -o square && "/Users/abhinavsharma/Comp. Language/C/"square

[Done] exited with code=null in 98.574 seconds

[Running] cd "/Users/abhinavsharma/Comp. Language/C/" && gcc hollowSquare.c -o hollowSquare && "/Users/abhinavsharma/Comp. Language/C/"hollowSquare

main*

Ln 13, Col 14

Spaces: 4

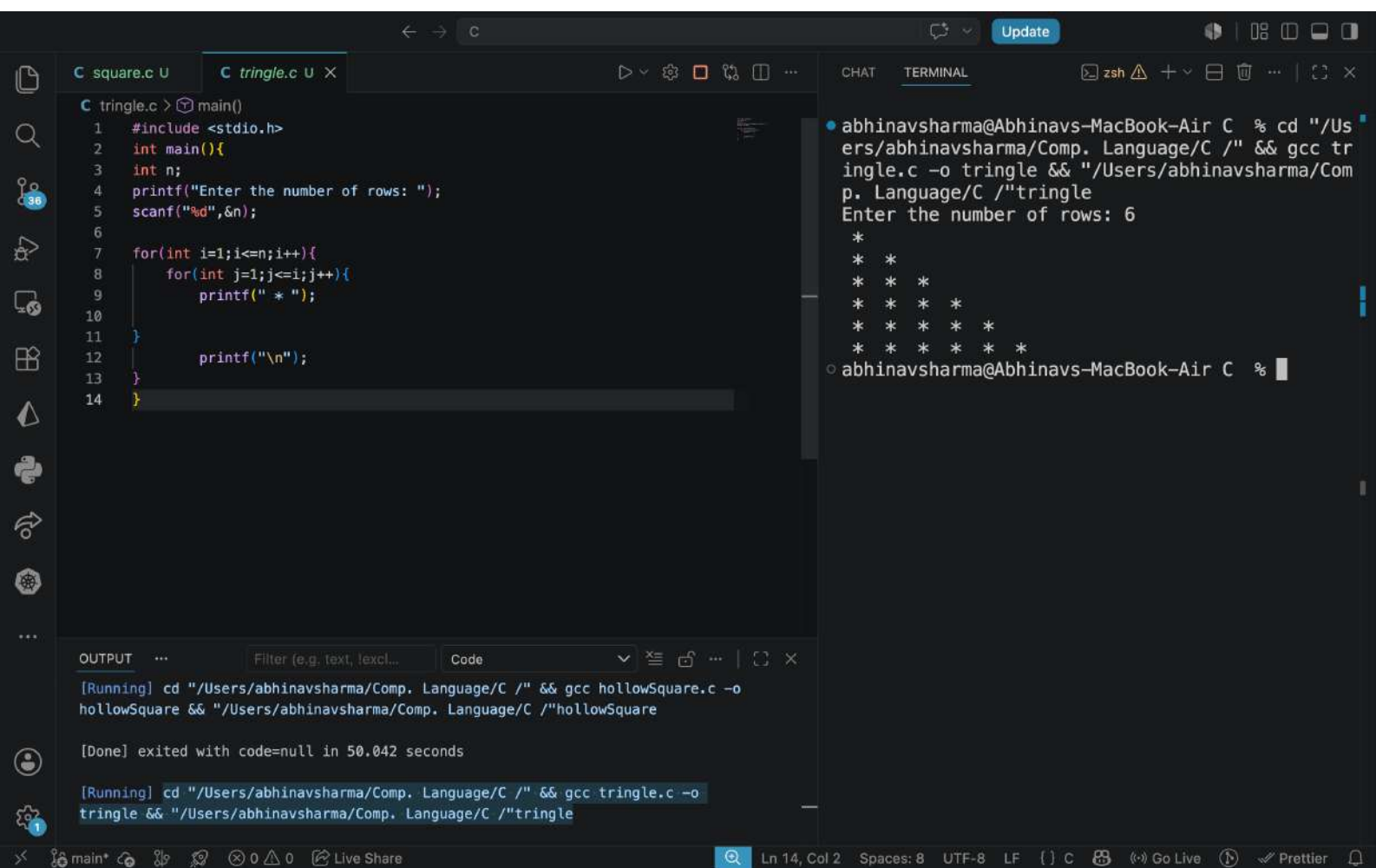
UTF-8

LF

{ } C

Go Live

Prettier



← → C

Update

zsh

C square.c U

C invertedtriangle.c U X

invertedtriangle.c > main()

```
1 #include<stdio.h>
2 int main(){
3     int n;
4     printf("Enter the number of rows: ");
5     scanf("%d",&n);
6     for(int i=n;i>=1;i--){
7         for(int j=1;j<=i;j++){
8             printf(" * ");
9         }
10        printf("\n");
11    }
12 }
13 }
```

CHAT

TERMINAL

abhinavsharma@Abhinavs-MacBook-Air C % cd "/Users/abhinavsharma/Comp. Language/C/" && gcc i

nvertedtriangle.c -o invertedtriangle && "/User

s/abhinavsharma/Comp. Language/C/"invertedtria

ngle

Enter the number of rows: 6

* * * * *
 * * * *
 * * *
 * *
 *

abhinavsharma@Abhinavs-MacBook-Air C %

OUTPUT

Filter (e.g. text, textcl...)

Code

[Running] cd "/Users/abhinavsharma/Comp. Language/C/" && gcc tringle.c -o

tringle && "/Users/abhinavsharma/Comp. Language/C/"tringle

[Done] exited with code=null in 38.816 seconds

[Running] cd "/Users/abhinavsharma/Comp. Language/C/" && gcc invertedtriangle.c -o

invertedtriangle && "/Users/abhinavsharma/Comp. Language/C/"invertedtriangle

main*

Ln 13, Col 2

Spaces: 8

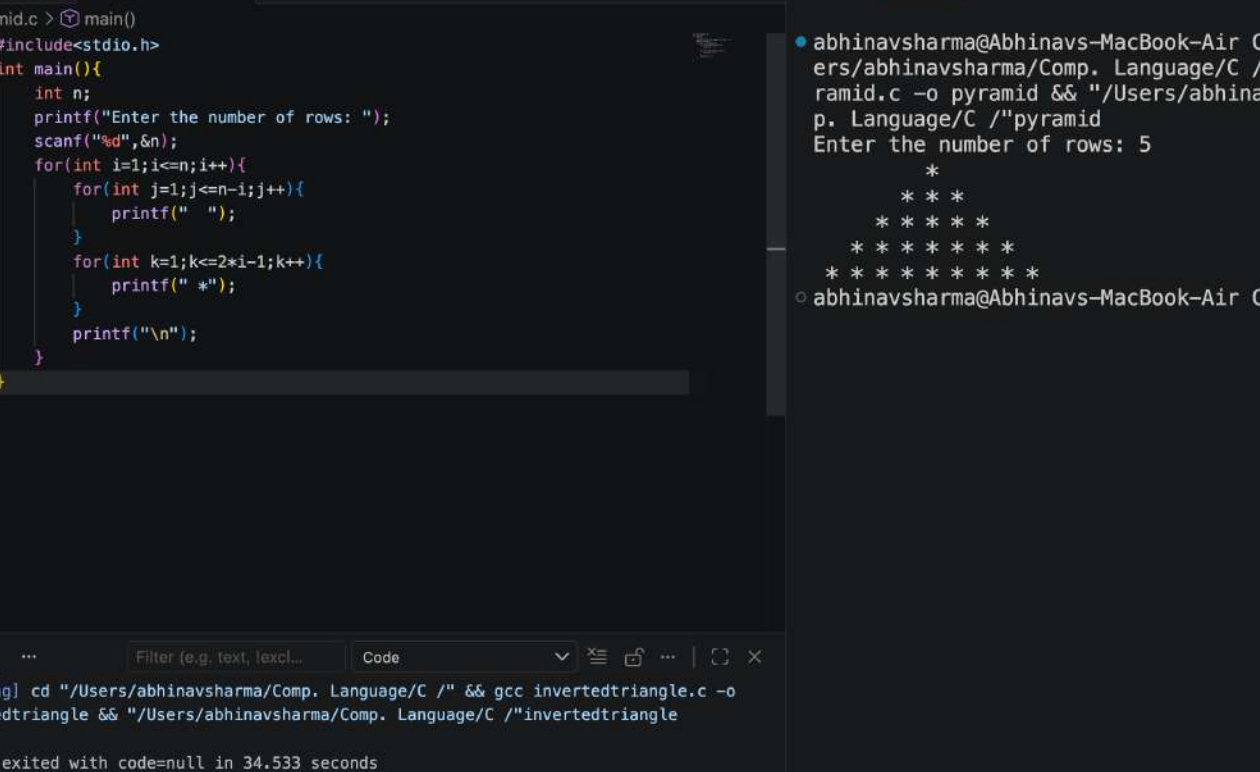
UTF-8

LF

{ } C

Go Live

Prettier



```
C square.c U C pyramid.c U x
C pyramid.c > main()
1 #include<stdio.h>
2 int main(){
3     int n;
4     printf("Enter the number of rows: ");
5     scanf("%d",&n);
6     for(int i=1;i<=n;i++){
7         for(int j=1;j<=n-i;j++){
8             printf(" ");
9         }
10        for(int k=1;k<=2*i-1;k++){
11            printf(" *");
12        }
13        printf("\n");
14    }
15 }
```

abhinavsharma@Abhinavs-MacBook-Air C % cd "/Users/abhinavsharma/Comp. Language/C/" && gcc pyramid.c -o pyramid && "/Users/abhinavsharma/Comp. Language/C/"pyramid

Enter the number of rows: 5

```

      *
     * *
    * * *
   * * * *
  * * * * *
```

abhinavsharma@Abhinavs-MacBook-Air C %

OUTPUT ... Filter (e.g. text, text...) Code

[Running] cd "/Users/abhinavsharma/Comp. Language/C/" && gcc invertedtriangle.c -o invertedtriangle && "/Users/abhinavsharma/Comp. Language/C/"invertedtriangle

[Done] exited with code=null in 34.533 seconds

[Running] cd "/Users/abhinavsharma/Comp. Language/C/" && gcc pyramid.c -o pyramid && "/Users/abhinavsharma/Comp. Language/C/"pyramid

```
C square.c U C invertedoyramid.c U X
C invertedoyramid.c > ...
1  #include<stdio.h>
2  int main(){
3      int n;
4      printf("Enter the number of rows: ");
5      scanf("%d",&n);
6      for(int i=n;i>=1;i--){
7          for(int k=1;k<=n-i;k++){
8              printf(" ");
9          }
10         for(int j=1;j<=2*i-1;j++){
11             printf("*");
12         }
13         printf("\n");
14     }
15     return 0;
16 }
```

OUTPUT ... Filter (e.g. text, text...) Code

[Running] cd "/Users/abhinavsharma/Comp. Language/C/" && gcc pyramid.c -o pyramid && "/Users/abhinavsharma/Comp. Language/C/"pyramid

[Done] exited with code=null in 70.919 seconds

[Running] cd "/Users/abhinavsharma/Comp. Language/C/" && gcc invertedoyramid.c -o invertedoyramid && "/Users/abhinavsharma/Comp. Language/C/"invertedoyramid

```
abhinavsharma@Abhinavs-MacBook-Air C % cd "/Users/abhinavsharma/Comp. Language/C/" && gcc invertedoyramid.c -o invertedoyramid && "/Users/abhinavsharma/Comp. Language/C/"invertedoyramid
Enter the number of rows: 6
*****
*****
*****
****
***
**
*
```


The screenshot displays a code editor interface with two tabs: "square.c U" and "halfdiamond.c U X". The active tab is "halfdiamond.c U X", which contains the following C code:

```
1 #include<stdio.h>
2 int main(){
3     int n;
4     printf("Enter the number of rows: ");
5     scanf("%d",&n);
6     for(int i=1;i<=n;i++){
7         for(int j=1;j<=i;j++){
8             printf(" *");
9         }
10        printf("\n");
11    }
12    for(int i=n-1;i>=1;i--){
13        for(int j=n;j>=n-i+1;j--){
14            printf(" *");
15        }
16        printf("\n");
17    }
18 }
19
20
21 }
```

The output panel at the bottom shows the execution results:

```
[Running] cd "/Users/abhinavsharma/Comp. Language/C /" && gcc invertedoyramid.c -o
invertedoyramid && "/Users/abhinavsharma/Comp. Language/C /"invertedoyramid

[Done] exited with code=null in 27.201 seconds

[Running] cd "/Users/abhinavsharma/Comp. Language/C /" && gcc halfdiamond.c -o
halfdiamond && "/Users/abhinavsharma/Comp. Language/C /"halfdiamond
```

The terminal window on the right side of the editor shows the command prompt and the output of the program:

```
abhinavsharma@Abhinavs-MacBook-Air C % cd "/Us
ers/abhinavsharma/Comp. Language/C /" && gcc ha
lfdiamond.c -o halfdiamond && "/Users/abhinavsh
arma/Comp. Language/C /"halfdiamond
Enter the number of rows: 7
 *
 * *
 * * *
 * * * *
 * * * * *
 * * * * *
 * * * * *
 * * * * *
 * * * *
 * * *
 * *
 *
```

```

C:\diamond.c > main()
1  #include <stdio.h>
2
3  int main() {
4      int n;
5
6      printf("Enter the number of rows: ");
7      scanf("%d", &n);
8      // Upper half
9      for(int i = 1; i <= n; i++) {
10         for(int j = 1; j <= n - i; j++)
11             printf(" ");
12         for(int k = 1; k <= 2 * i - 1; k++)
13             printf("*");
14         printf("\n");
15     }
16     // Lower half
17     for(int i = n - 1; i >= 1; i--) {
18         for(int j = 1; j <= n - i; j++)
19             printf(" ");
20         for(int k = 1; k <= 2 * i - 1; k++)
21             printf("*");
22         printf("\n");
23     }
24     return 0;
25 }

```

```
• abhinavsharma@Abhinavs-MacBook-Air C % cd "/Users/abhinavsharma/Comp. Language/C/" && gcc diamond.c -o diamond && "/Users/abhinavsharma/Comp. Language/C/"diamond
Enter the number of rows: 7
```

```
*  
***  
*****  
*******  
*****  
***  
**  
*  
**  
***  
****  
*****
```

```
abhinavsharma@Abhinavs-MacBook-Air C %
```